

Benchmark OpenMP: Memory-bound e Compute-bound

Aluno: Thiago Jordão

Objetivo

Este trabalho implementa dois programas paralelos em C com OpenMP para comparar o comportamento de aplicações limitadas por memória e por processamento.

- **Memory-bound:** soma simples de vetores, com acesso sequencial aos dados.
- **Compute-bound:** cálculo matemático intensivo, com muitas operações por elemento.

A proposta é variar o número de threads, medir o tempo de execução e analisar quando o desempenho melhora, estabiliza ou piora.

Estrutura dos programas

1. `memory_bound.c`

Esse programa executa a soma de dois vetores e grava o resultado em um terceiro vetor. A parte paralela usa:

```
#include <math.h>
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define DEFAULT_N 100000000
#define DEFAULT_MAX_THREADS 8

static double seconds_since(double start) {
    return omp_get_wtime() - start;
}

static void fill_vectors(double *a, double *b, double *c, size_t n) {
    #pragma omp parallel for
    for (size_t i = 0; i < n; i++) {
        a[i] = (double)(i % 100) * 0.25;
        b[i] = (double)(i % 50) * 0.5;
        c[i] = 0.0;
    }
}

static double vector_sum(double *a, double *b, double *c, size_t n) {
    double checksum = 0.0;

    #pragma omp parallel for reduction(+:checksum)
    for (size_t i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}
```

```
        checksum += c[i];
    }

    return checksum;
}

int main(int argc, char **argv) {
    size_t n = DEFAULT_N;
    if (argc > 1) {
        n = (size_t)strtoull(argv[1], NULL, 10);
    }
    if (n == 0) {
        fprintf(stderr, "Uso: %s [tamanho]\n", argv[0]);
        return 1;
    }

    double *a = (double *)malloc(n * sizeof(double));
    double *b = (double *)malloc(n * sizeof(double));
    double *c = (double *)malloc(n * sizeof(double));
    if (!a || !b || !c) {
        fprintf(stderr, "Erro: falha na alocação de memória.\n");
        free(a);
        free(b);
        free(c);
        return 1;
    }

    int threads = omp_get_max_threads();
    printf("Benchmark memory-bound: soma de vetores\n");
    printf("Tamanho: %zu\n", n);
    printf("Threads: %d\n", threads);

    fill_vectors(a, b, c, n);

    double start = omp_get_wtime();
    double checksum = vector_sum(a, b, c, n);
    double elapsed = seconds_since(start);

    printf("Tempo(s): %.6f  Soma: %.2f\n", elapsed, checksum);

    free(a);
    free(b);
    free(c);
    return 0;
}
```

Como a operação por elemento é pequena e o acesso à memória é dominante, espera-se ganho limitado por largura de banda de memória e por custo de sincronização.

2. `compute_bound.c`

Esse programa aplica uma função matemática custosa em cada elemento de um vetor. A parte paralela também usa:

```
#include <math.h>
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define DEFAULT_N 20000000
#define DEFAULT_MAX_THREADS 8

static double heavy_math(double x) {
    double value = x;

    for (int k = 0; k < 40; k++) {
        value = sin(value) * cos(value) + sqrt(fabs(value) + 1.0);
        value = value * 0.999999 + 0.000001 * x;
    }

    return value;
}

static double compute_kernel(double *input, size_t n) {
    double checksum = 0.0;

    #pragma omp parallel for reduction(+:checksum)
    for (size_t i = 0; i < n; i++) {
        double x = input[i];
        double result = heavy_math(x);
        checksum += result;
    }

    return checksum;
}

int main(int argc, char **argv) {
    size_t n = DEFAULT_N;
    if (argc > 1) {
        n = (size_t)strtoull(argv[1], NULL, 10);
    }
    if (n == 0) {
        fprintf(stderr, "Uso: %s [tamanho]\n", argv[0]);
        return 1;
    }

    double *input = (double *)malloc(n * sizeof(double));
    if (!input) {
        fprintf(stderr, "Erro: falha na alocao de memoria.\n");
        return 1;
    }

    #pragma omp parallel for
```

```
for (size_t i = 0; i < n; i++) {
    input[i] = (double)(i % 1000) * 0.001 + 0.5;
}

int threads = omp_get_max_threads();
printf("Benchmark compute-bound: calculo matematico intensivo\n");
printf("Tamanho: %zu\n", n);
printf("Threads: %d\n", threads);

double start = omp_get_wtime();
double checksum = compute_kernel(input, n);
double elapsed = omp_get_wtime() - start;

printf("Tempo(s): %.6f Soma: %.2f\n", elapsed, checksum);

free(input);
return 0;
}
```

Aqui o custo computacional por iteração é alto, então o paralelismo tende a escalar melhor até o ponto em que a competição por recursos internos do processador começa a dominar.

Como compilar e executar

```
make
./memory_bound [tamanho] [threads_max]
./compute_bound [tamanho] [threads_max]
```

Exemplos:

```
./memory_bound 100000000 8
./compute_bound 20000000 8
```

Metodologia de medição

1. Executar cada programa com diferentes números de threads, de 1 até o máximo definido.
2. Registrar o tempo de execução de cada configuração.
3. Comparar o comportamento entre as duas classes de problema.

Ambiente de execução (CPU)

Saída relevante do `lscpu`:

- **CPU(s): 8** (CPUs lógicas disponíveis)
- **Thread(s) per núcleo: 2**
- Portanto: **4 núcleos físicos** e **SMT 2x** (4C/8T)

Interpretação para os testes:

- **1–4 threads:** tende a escalar usando núcleos físicos distintos.
- **5–8 threads:** passa a usar SMT (2 threads competindo dentro do mesmo núcleo).

Tabela de resultados

Memory-bound

Tamanho: 20000000

Threads	Tempo (s)	Speedup
1	0.045000	1.000
2	0.045000	1.000
4	0.042000	1.071
6	0.036000	1.250
8	0.035000	1.286
12	0.040000	1.125
16	0.040000	1.125

Compute-bound

Tamanho: 20000000

Threads	Tempo (s)	Speedup
1	121.343000	1.000
2	60.750000	1.997
4	38.270000	3.171
6	31.864000	3.808
8	33.987000	3.570
12	30.533000	3.974
16	28.970000	4.189

Análise (observada)

No compute-bound, houve ganho expressivo até 4 threads (núcleos físicos) e o desempenho continuou melhorando até 8 threads, indicando que o SMT também ajudou neste caso específico. Isso pode ocorrer quando parte do tempo é perdida com latências (ex.: funções matemáticas transcendentais) e o SMT consegue manter as unidades ocupadas.

Ainda assim, em workloads compute-bound mais “puros” (ex.: muitas operações vetoriais/FMA sem stalls), é comum o ganho após 4 threads reduzir bastante ou até piorar, pois 2 threads no mesmo núcleo competem por unidades de execução, registradores, cache e largura de banda interna.

Reflexão

O multithreading de hardware tende a ajudar mais em programas memory-bound quando uma thread fica esperando dados da memória e outra pode ocupar a execução. Já em programas compute-bound, as threads competem mais diretamente pelos recursos do processador, o que pode reduzir ou até anular o ganho.

Conclusão

Os dois benchmarks permitem observar que OpenMP é útil, mas o ganho depende fortemente do perfil da aplicação. Programas com muito acesso à memória tendem a saturar rapidamente. Programas com muita conta matemática escalam melhor, até o ponto em que a disputa por recursos e o overhead paralelizam o custo em vez de reduzi-lo.